# Algorithme retenu - Classification de papillons

## Algorithme retenu : Classification de papillons (LPU)

### 1. Dataset

Le dataset utilisé est **Butterfly Images 40 Species** (`gpiosenka/butterfly-images40-species`, Kaggle), malgré son nom, il compte aujourd'hui **100 espèces**. Contrairement au dataset utilisé initialement (Smithsonian Butterflies, HuggingFace), les classes sont ici propres et suffisamment représentées : pas besoin de regrouper par genre, on classe directement l'**espèce**. Le dataset fournit ses propres splits `train/` (13094 images) et `test/` (500 images, 5 par classe) ; `valid/` est fusionné avec `train/` pour l'entraînement.

### 2. Algorithme retenu : transfer learning (CNN gelé + classifieur linéaire)

**Extraction de features : MobileNetV2 pré-entraîné sur ImageNet, gelé** Chaque image est passée dans MobileNetV2 (poids ImageNet, `torchvision`) dont la tête de classification est retirée : on récupère l'avant-dernière couche, un vecteur de 1280 dimensions qui résume le contenu visuel de l'image. Aucun entraînement du CNN n'est effectué (pas de rétropropagation) : uniquement des passes forward, ce qui reste utilisable sur CPU.

**Normalisation : `StandardScaler`**, appliqué sur les embeddings avant le classifieur (indispensable pour la convergence d'un SGD).

**Classifieur : `SGDClassifier`** (perte `log_loss`), entraîné par `partial_fit` sur 15 epochs, avec ré-mélange des données à chaque epoch.

#### Justification par rapport aux alternatives recensées (`eval1/Research.md`)

Un premier prototype a été construit avec **HOG + histogramme de couleur HSV + `SGDClassifier`**, l'option "Mix" la plus légère recensée dans `Research.md`. Il tourne entièrement sur CPU sans dépendance lourde, mais plafonne à 27,4 % de Top-1 (voir tableau en section 4) : des features géométriques et colorimétriques faites à la main ne suffisent pas à distinguer 100 espèces visuellement proches.

`Research.md` écarte l'entraînement d'un CNN/Transformer *from scratch* pour ce prototype (pas de GPU, volume de données par classe trop faible pour éviter le sur-apprentissage). Mais le **transfer learning** (utiliser un CNN déjà entraîné sur ImageNet comme extracteur de features fixe, sans le ré-entraîner) permet de bénéficier de la puissance des architectures profondes de `Research.md` (ResNet, MobileNet…) sans ni GPU ni gros volume de données : seul le classifieur final (`SGDClassifier`, quelques dizaines de milliers de paramètres) est entraîné. C'est le compromis retenu : la partie coûteuse (apprentissage des features visuelles génériques) a déjà été faite par les auteurs de MobileNetV2 sur ImageNet ; il ne reste plus qu'à apprendre la frontière de décision entre les 100 espèces à partir de ces features, tâche largement à la portée d'un classifieur linéaire entraîné sur CPU.

MobileNetV2 est préféré à un ResNet plus profond pour sa légèreté (inférence rapide sur CPU, ~14 Mo de poids), suffisante puisque le réseau ne sert que de descripteur d'image, pas de classifieur final.

## 3. Pipeline

```
image brute (dataset Kaggle)
   │
   ▼
MobileNetV2 (ImageNet, gelé) → embedding 1280-d   (src/pretreatment.py:pretreat)
   │
   ▼
StandardScaler
   │
   ▼
SGDClassifier (log_loss), partial_fit, 15 epochs   (main.py:train)
   │
   ▼
model.joblib  (clf + scaler)
   │
   ▼
predict_proba sur le split de test → top-5 par espèce   (predict.py)
   │
   ▼
predictions.csv (true_label, top1..top5)
   │
   ▼
eval2/evaluate.py → Top-1, Top-5, Macro F1, matrice de confusion
```

L'extraction de features est parallélisée sur les cœurs CPU disponibles (`joblib.Parallel`).

## 4. Résultats obtenus

Évaluation sur le split de test (500 images, 100 espèces), critères définis dans `eval2/criteres_evaluation.md` ; détails complets dans `eval3/resultats/` :

| Approche | Top-1 | Top-5 | Macro F1 |
|---|---|---|---|
| Ancien dataset (Smithsonian, genre, 85 classes) | 24,74 % | 59,58 % | 0,19 |
| HOG + couleur + SGD (prototype intermédiaire) | 27,4 % | 36,4 % | 0,33 |
| **MobileNetV2 (gelé) + SGD (retenu)** | **75,0 %** | **81,0 %** | **0,806** |

Le transfer learning place le prototype dans la fourchette "modèle correct/fort" de `eval2/criteres_evaluation.md` (Top-1 attendu 70-85 % pour un CNN fine-tuné), alors que l'approche purement classique restait "modèle faible" (< 60 %). La précision moyenne (macro avg precision = 0,94) montre que le modèle se trompe rarement quand il est confiant ; le rappel plus variable selon les classes (0,20 à 1,00) indique que certaines espèces sont encore régulièrement manquées.

**Liste de tests (extrait, détail complet dans `eval3/resultats/scores_par_classe.csv`)**

| Classe | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| AFRICAN GIANT SWALLOWTAIL | 1.00 | 1.00 | 1.00 | 5 |
| GIANT LEOPARD MOTH | 1.00 | 1.00 | 1.00 | 5 |
| LUNA MOTH | 1.00 | 1.00 | 1.00 | 5 |
| ORANGE TIP | 1.00 | 1.00 | 1.00 | 5 |
| ADONIS | 0.05 | 1.00 | 0.09 | 5 |
| AMERICAN SNOOT | 1.00 | 0.20 | 0.33 | 5 |
| CINNABAR MOTH | 1.00 | 0.20 | 0.33 | 5 |
| GREAT EGGFLY | 1.00 | 0.20 | 0.33 | 5 |
| SLEEPY ORANGE | 1.00 | 0.20 | 0.33 | 5 |

`ADONIS` est un cas particulier : précision très faible malgré un rappel parfait : le modèle y classe par défaut plusieurs espèces qu'il ne reconnaît pas ailleurs (voir `eval3/resultats/confusion_matrix.png`).

## 5. Limites et perspectives

- **Dépendance lourde** : `torch`/`torchvision` (quelques centaines de Mo, y compris des paquets CUDA inutilisés puisqu'on tourne en CPU), à comparer à la légèreté de HOG (`scikit-image` seul).
- **Classe "fourre-tout"** (`ADONIS`) : un seuil de confiance ou une classe "rejet" pourrait réduire les faux positifs plutôt que de forcer une prédiction parmi les 100 classes.
- **Pistes d'amélioration** : dégeler les dernières couches de MobileNetV2 (fine-tuning léger) plutôt que de le garder entièrement figé ; essayer un backbone plus profond (ResNet18/EfficientNet) si le temps d'inférence CPU le permet encore ; recherche d'hyperparamètres (`alpha`, nombre d'epochs) sur le `SGDClassifier`.